Headless CMS

Technical Documentation ENG

Strapi Backend • SvelteKit Frontend

Responsible IT
June 2026

Table of Contents

1. Introduction & Project Overview

This document describes the **Headless CMS** built by responsibleIT. It consists of two separate repositories that work together: a **Strapi backend** that manages and exposes content through a REST API, and a **SvelteKit frontend** that fetches that content at build time to produce a fully static website.

The architecture is “headless” because the CMS (Strapi) has no built-in website — it only provides data via an API. The visual website is built and maintained separately in SvelteKit, consuming that API.

Repositories

Backend — Strapi CMS	
Repo	responsibleIT/test-website-strapi
Branch	main
Technology	Strapi v5 (Node.js / TypeScript)
Default port	1337
Database	SQLite
Purpose	Content management, REST API, media uploads, deploy webhook

Frontend — SvelteKit	
Repo	responsibleIT/headless-test-website-strapi
Branch	svelte-rewrite
Technology	SvelteKit + Svelte 5 (static adapter)
Output	Fully pre-rendered static HTML/CSS/JS
Purpose	Fetch Strapi data at build time and render public website

How the two projects connect

At build time, SvelteKit calls the Strapi REST API to fetch all content. It generates one HTML file per page and outputs a static folder that can be deployed to any web host (In our case Coolify). At runtime (when a visitor loads the site) there is no connection to Strapi at all — the visitor only receives the pre-built static files.

> **Deploy on publish**
> When a content editor publishes or updates content in Strapi, Strapi fires a deploy webhook to Coolify which triggers a new build of the SvelteKit frontend, making the changes visible on the live site. (In our case by clicking on the small rocket)

2. System Architecture

Architecture diagram

Flow

Content Editor → Strapi Admin Panel → Strapi REST API → SvelteKit build → Static files → Visitor browser

Layer	What it does	Technology
Content Type builder	Visual definition of content types and components.	Strapi
Content management	Editors create/edit/publish content	Strapi
Data storage	Persists all content and media	SQLite
API	Exposes content as JSON via REST	Strapi REST API (port 1337)
Build	Fetches API and generates static HTML	SvelteKit + adapter-static
Hosting	Serves static files to visitors	Coolify
CI trigger	Kicks off new build on content change	Coolify deploy webhook

Fallback / local development

The frontend supports two API endpoints: a primary remote Strapi (`PUBLIC_STRAPI_API_URL`) and a local fallback (`PUBLIC_LOCAL_API_URL`). If the remote Strapi is unreachable (e.g. during local development), the build automatically retries against the local instance. This lets developers work offline with a local Strapi without changing any code.

3. Backend — Strapi CMS

What Strapi does

Strapi is the “brain” of the content system. It provides:

- A visual admin panel at /admin where editors manage all content
- A REST API at /api/* that the frontend queries at build time
- Media management — file uploads stored in /public/uploads/
- User & role management — controls who can access the API and with what permissions
- Draft & publish workflow — content can be saved as draft before going live
- A deploy webhook endpoint that triggers a frontend rebuild in Coolify (rocket)

Directory structure

```
strapi-backend/
├── config/              ← Server, database, plugins, middleware configuration
│   ├── database.ts      ← DB connection (SQLite dev, Postgres/MySQL prod)
│   ├── server.ts        ← Host and port settings
│   ├── plugins.ts       ← Enabled plugins (code editor, upload settings)
│   └── middlewares.ts   ← CORS, security headers, logging
├── src/
│   ├── api/             ← One folder per content type
│   │   ├── page/                    ← Generic CMS page
│   │   ├── navigation/              ← Site navigation (single type)
│   │   ├── partner/                 ← Partner organisations
│   │   ├── research-detail-page/    ← Research project entries
│   │   ├── student-detail-page/     ← Student project entries
│   │   └── rebuild/                 ← Custom deploy webhook endpoint
│   ├── components/      ← Reusable field groups used inside content types
│   │   ├── component/   ← Bronnen, Buttons, Onderwijseenheid, ResearcherData
│   │   ├── layout/      ← Section (page layout blocks)
│   │   └── navigation/  ← NavItems
│   └── index.ts         ← Strapi lifecycle hooks (register / bootstrap)
├── public/uploads/      ← Media files uploaded through the admin
└── types/generated/     ← Auto-generated TypeScript types (do not edit by hand)
```

Content types

A content type is a structured data model — like a database table — that editors fill in through the admin panel.

Page (collection type)

Generic website pages used for the homepage and any other flat content pages.

Field	Type	Description
title	String (required)	The page's display name
slug	UID → title	URL-safe identifier, auto-generated from title (click on reload when you change the value)
metaDescription	Text	SEO description shown in search results
video	Media (file/video)	Optional video attachment
sections	Component (Section, repeatable)	List of layout sections on the page

Navigation (single type)

There is only one Navigation record — it defines the site-wide navigation menu.

Field	Type	Description
navItems	Component (navItems, repeatable)	Ordered list of nav links (label, url, order)

Partner (collection type)

Organisations that collaborate on student or research projects.

Field	Type	Description
name	String (required)	Partner organisation name
student_detail_page	Relation (many-to-many)	Student projects this partner is linked to
research_detail_pages	Relation (many-to-many)	Research projects this partner is linked to

Student Detail Page (collection type)

One entry per student project. These appear under /student-projecten/ on the website.

Field	Type	Notes
ProjectTitel	String (required)	Project title, min 2 characters
Slug	UID → ProjectTitel	Auto URL slug, used in the page URL (click on reload when you change the value)
ProjectBeschrijving	String	Short description
LesJaar	Integer (0–99)	Academic year number, default 26
AantalStudenten	Integer (min 1)	Number of students in the project
Opleiding	Multi-select	Study programme(s): HBO-ICT, CMD, CO, CB, FDND
SchoolJaar	Multi-select	Year of study: 1, 2, 3 or 4
Niveau	Multi-select (max 1)	Level: Bachelor, Master, or AD
Doorlooptijd	String	Project duration (free text, e.g. “8 weken”)
VideoEmbed	String	Embed URL for a video (e.g. YouTube iframe src)
Foto	Media (image/file)	Project photograph
Audio	Media (multiple, audio)	One or more audio files
Onderwijseenheid	Component	Educational unit with label and URL
Bronnen	Component (repeatable)	Source/reference URLs
Partners	Relation (many-to-many)	Partner organisations
P5	Code editor (JS)	Optional P5.js sketch embed code

Research Detail Page (collection type)

One entry per research project. These appear under /research-projecten/ on the website.

Field	Type	Notes
ProjectTitel	String (required, unique)	Research project title
Slug	UID → ProjectTitel	Auto URL slug (click on reload when you change the value)
ProjectBeschrijving	Text	Project description
Jaar	Integer	Year the research was conducted
Duur	Integer	Duration (in some unit — e.g. months)
Regeling	String	Funding arrangement or programme
VideoEmbed	String	Embed URL for a video
Foto	Media (image/file)	Project photograph
Audio	Media (audio/file)	Audio file
ReasearcherData	Component (repeatable)	Researcher name + email pairs
Bronnen	Component (repeatable)	Source/reference URLs
partners	Relation (many-to-many)	Partner organisations
research_pages	Self-relation (many-to-many)	Related research projects
P5	Code editor (JS)	Optional P5.js sketch embed code

Components

Components are reusable field groups. Unlike content types, they have no API endpoint of their own — they only exist inside a parent content type.

Component	Fields	Used in
Section (layout)	type (hero/text/cta/gallery), heading, body, image, buttons[]	Page
Buttons (component)	label, url, style (primary/secondary/outline), openInNewTab	Section
navItems (navigation)	label, url, order	Navigation
Bronnen (component)	Bron (unique string URL)	Student + Research pages
Onderwijseenheid (component)	label, URL	Student Detail Page
ReasearcherData (component)	Naam, Email	Research Detail Page

Plugins installed

Plugin	Purpose
strapi-plugin-multi-select	Adds a multi-select field type (used for Opleiding, SchoolJaar, Niveau)
strapi-code-editor-custom-field	Adds an in-admin code editor field (used for P5 sketches)
strapi-plugin-config-sync	Syncs plugin configuration to/from JSON files in version control
@strapi/plugin-users-permissions	Manages public/authenticated API access permissions
@strapi/plugin-cloud	Optional Strapi Cloud deployment support

Custom endpoint: /api/rebuild

The `src/api/rebuild/` folder contains a custom POST endpoint. When called, it reads the `COOLIFY_DEPLOY_WEBHOOK` environment variable and makes a GET request to Coolify's deploy webhook URL to trigger a new build of the SvelteKit frontend.

This endpoint has no authentication (`auth: false`), so it is designed to be called by Strapi's built-in webhook system after a content-publish event. The token is extracted from the webhook URL itself and sent as a Bearer header.

Configuration files

config/database.ts

Supports SQLite (default for development), PostgreSQL, and MySQL. The active database is selected via the `DATABASE_CLIENT` environment variable. SQLite stores its file at `.tmp/data.db`.

config/server.ts

Binds to `0.0.0.0` on port `1337` by default. Overridable via `HOST` and `PORT` environment variables.

config/middlewares.ts

Enables standard Strapi middleware stack. Adds a custom Content Security Policy to allow Monaco editor scripts from `cdn.jsdelivr.net` (required by the code editor plugin).

config/plugins.ts

Enables `strapi-code-editor-custom-field` and configures image breakpoints for the upload plugin (xsmall: 64px → xlarge: 1920px).

4. Frontend — SvelteKit

What SvelteKit does

SvelteKit is the framework that turns Strapi's API data into a public website. Key characteristics:

- **Static site generation (SSG):** every page is pre-rendered to plain HTML at build time. No server-side code runs when a visitor opens the site.
- **Svelte 5 runes mode:** the project uses Svelte's latest reactivity model (`$props()`, `$state()`, `$derived()`) throughout.
- **adapter-static:** the build output is a folder of HTML/CSS/JS files deployable to any static host.
- **Fallback 404.html:** the adapter is configured with a fallback page for client-side routing on hosts that support it.

Directory structure

```
svelte-frontend/src/
├── app.html                ← HTML shell (head, body, favicon, reveal script)
├── lib/
│   ├── assets/             ← favicon.svg
│   ├── components/
│   │   ├── PageCard/       ← Expandable card showing a Page with its sections
│   │   └── SectionCard/    ← Renders a single Section component block
│   └── index.js            ← (empty barrel file)
├── routes/
│   ├── +layout.js          ← Sets prerender=true globally
│   ├── +layout.svelte      ← Root layout: imports global CSS, favicon, keyboard shortcut
│   ├── +page.server.js     ← Fetches home page + all pages list from Strapi
│   ├── +page.svelte        ← Renders home page with sections and page list
│   ├── [slug]/             ← Dynamic route for generic CMS pages
│   │   ├── +page.server.js    ← Fetches one page by slug + generates prerender entries
│   │   └── +page.svelte       ← Renders a single page
│   ├── research-projecten/
│   │   ├── +page.server.js    ← Fetches all research projects
│   │   ├── +page.svelte       ← Lists all research projects
│   │   └── [slug]/            ← Research project detail page
│   └── student-projecten/
│       ├── +page.server.js    ← Fetches all student projects
│       ├── +page.svelte       ← Lists all student projects
│       └── [slug]/            ← Student project detail page
└── styles/
    ├── general.css         ← Global reset, body font, layout utilities
    └── pages/home.css      ← Home-page specific decorative styles
```

Routes

URL pattern	Content source	Notes
/	Page with slug “home” + all pages	Landing page with section renderer
/[slug]	Page collection (by slug)	Any generic CMS page; reserved slugs excluded
/student-projecten	student-detail-pages collection	Lists all student projects
/student-projecten/[slug]	student-detail-pages (by Slug)	Full student project detail
/research-projecten	research-detail-pages collection	Lists all research projects
/research-projecten/[slug]	research-detail-pages (by Slug)	Full research project detail

The slugs student-projecten, research-projecten, and home are “reserved” — the [slug] catch-all route explicitly skips them so they are handled by their own dedicated route folders.

Environment variables

Create a .env file in the svelte-frontend root with these variables:

```
# Remote Strapi (used first)
PUBLIC_STRAPI_API_URL='http://gwtti5qjvak0lg9nots6p1uk.185.88.141.173.sslip.io'

LOCAL_API_TOKEN=ce8d69900f72bc4890ebd686038f347fa9c7414fea6a00e9721524174c1d588f66c639d2
9fb82ddc32e5530d02068762dff14794c22524af569dd97ed7b9ff25fa901e672ea0611b45c9227acc6e6389
3ff3241bca3d36e9c500dc84c4e9a622a122cb4e81ea9d4fbb7fef63451f8c68e808303e246c55a52670d4c0
5caeaebc

# Local Strapi fallback (used if remote is unavailable)
PUBLIC_LOCAL_API_URL=http://localhost:1337
LOCAL_API_TOKEN=your_local_strapi_api_token_here
```

Variable	Scope	Description
PUBLIC_STRAPI_API_URL	Public (exposed to browser)	Base URL of the primary Strapi instance. Also used to prefix media file URLs.
API_TOKEN	Private (build server only)	Strapi API token for the remote instance.
PUBLIC_LOCAL_API_URL	Public	Base URL of the local/fallback Strapi instance.
LOCAL_API_TOKEN	Private	Strapi API token for the local instance.

Important

PUBLIC_ variables are embedded in the built JavaScript and visible to end-users. Never put secrets in PUBLIC_ variables. The API_TOKEN variables are private (used only at build time by +page.server.js files) and are not included in the output.

Data fetching pattern

Every +page.server.js file follows the same fetch pattern:

- **Primary:** Attempt the remote Strapi URL with API_TOKEN.
- **Fallback:** If the primary fails, retry against the local Strapi URL with LOCAL_API_TOKEN.
- **Graceful degradation:** Return null or an empty array if both fail, so the page can render a meaningful empty state rather than crashing the build.

Pre-rendering (entries() functions)

For dynamic routes like `[slug]`, SvelteKit needs to know in advance which URLs exist so it can pre-render them. Each dynamic route exports an `entries()` function that fetches the list of all records from Strapi and returns an array of slug objects. SvelteKit then pre-renders one HTML file per slug.

```
// Example from [slug]/+page.server.js
export async function entries() {
  const json = await fetchWithFallback('/api/pages', { 'pagination[pageSize]': '100' });
  return json.data
    .filter(p => p.slug && !RESERVED_SLUGS.has(p.slug))
    .map(p => ({ slug: p.slug }));
}
```

Svelte components

SectionCard

Renders a single `layout.section` component block. Displays the section type badge, an optional image (fetched from Strapi using the full base URL), and a key-value list of all non-ignored fields. Ignores `id`, `__component`, `type`, `image`, and `buttons` (these are rendered separately).

PageCard

An expandable card component shown on the home page. Displays page title (as a link), meta description, section count badge, video badge, and published status. Clicking the card expands it to show all its SectionCard children. Used as a developer-facing overview.

The “revealed” shortcut

The layout includes a hidden keyboard shortcut: pressing Enter, Enter, Backspace in sequence adds a .revealed class to <body>. This reveals decorative body::before / body::after pseudo-elements defined in home.css. The revealed state persists in localStorage for 3 days. This is intentionally undocumented in the UI — it's a developer/client preview feature.

5. Getting Started — Local Setup

Prerequisites

- Node.js 20 or higher
- pnpm (or npm / yarn)
- Git

1. Set up the Strapi backend

```
# Clone the backend
git clone https://github.com/responsibleIT/test-website-strapi.git
cd test-website-strapi

# Install dependencies
pnpm install

# Copy environment file and fill in values
cp .env.example .env    # (or create .env manually — see section 5.3)

# Start in development mode (with hot reload)
pnpm dev

# Strapi is now running at http://localhost:1337
# Admin panel: http://localhost:1337/admin
```

2. Set up the SvelteKit frontend

```
# Clone the frontend (svelte-rewrite branch)
git clone -b svelte-rewrite https://github.com/responsibleIT/headless-test-website-
strapi.git
cd headless-test-website-strapi

# Install dependencies
pnpm install

# Create environment file
cat > .env << EOF
PUBLIC_STRAPI_API_URL=https://your-remote-strapi.example.com
API_TOKEN=your_remote_token
PUBLIC_LOCAL_API_URL=http://localhost:1337
LOCAL_API_TOKEN=your_local_token
EOF

# Start the dev server (runs build-time fetches on every page load)
pnpm dev

# Site is at http://localhost:5173
```

3. Required Strapi environment variables

Create a .env file in the strapi-backend folder:

```
HOST=0.0.0.0

PORT=1337

# Secrets

APP_KEYS=pQRX68rofI5LbnNh0xyDww==,4aJug/yBQ1wacVKd/qRc3A==,5kGNxLeYkxT04mBVZSuatQ==,e7ng
02tL6yitGeRH9ciQiw==

API_TOKEN_SALT=MrQVoexZGRNbJ+drRWFlUw==

ADMIN_JWT_SECRET=r00YTsaE8yri1ulskfekvQ==

TRANSFER_TOKEN_SALT=jjGZ7iz2yfo9Ijyw39bSQQ==

ENCRYPTION_KEY=uwGWOYjbEckHPGwldYXeWA==

# Database

DATABASE_CLIENT=sqlite

DATABASE_HOST=

DATABASE_PORT=

DATABASE_NAME=

DATABASE_USERNAME=

DATABASE_PASSWORD=

DATABASE_SSL=false

DATABASE_FILENAME=.tmp/data.db

JWT_SECRET=iOEuelvCShkjp1NTgQJ3Lw==
```

4. Creating a Strapi API token

- Go to http://localhost:1337/admin → Settings → API Tokens
- Click “Create new API token”
- Give it a name, set type to “Read-only” (or “Full access” for local dev)
- Copy the token and put it in the frontend's .env as API_TOKEN or LOCAL_API_TOKEN
- In Settings → Users & Permissions → Roles → Public: enable “find” and “findOne” on each content type the frontend needs

6. Deployment

Overview

Both projects are deployed to Coolify (a self-hosted PaaS). The Strapi backend runs as a persistent Node.js server; the SvelteKit frontend is built as a static site and served as files.

Strapi backend deployment

- Push the strapi-backend repo to your Git remote.
- In Coolify, create a new Application pointing to the repo.
- Set the build command to `pnpm build` and the start command to `pnpm start`.
- Add all required environment variables (see section 5.3).
- For production, set DATABASE_CLIENT to postgres or mysql and configure the connection.
- Deploy. Strapi will be accessible at your chosen domain on port 1337 (or behind a reverse proxy on port 443).

SvelteKit frontend deployment

- Push the headless-test-website-strapi repo (svelte-rewrite branch) to your Git remote.
- In Coolify, create a Static Site Application pointing to the repo.
- Set the build command to `pnpm build`.
- Set the publish directory to `build`.
- Add the environment variables (PUBLIC_STRAPI_API_URL, API_TOKEN, PUBLIC_LOCAL_API_URL, LOCAL_API_TOKEN).
- Deploy. After a successful build, the site is live.

Automated rebuild on content publish

When a content editor publishes new content in Strapi, the static site needs to be rebuilt. This is handled automatically:

- **Strapi webhook:** In the Strapi admin go to Settings → Webhooks → Create new webhook.
- **Event:** Select “Entry → Publish”.
- **URL:** Point it to your Strapi API: https://your-strapi.example.com/api/rebuild
- **Coolify webhook:** The rebuild controller reads COOLIFY_DEPLOY_WEBHOOK and calls Coolify's deploy API, which starts a new SvelteKit build.
- **Result:** Within a few minutes, the live site reflects the new content.

7. Content Editor Guide

Logging in

Go to your Strapi admin panel URL (e.g. https://cms.yoursite.com/admin). Log in with your credentials. If you don't have an account, ask a developer or admin to create one under Settings → Users.

Creating a new student project

- In the left sidebar, click Content Manager → Student detail page.
- Click “Create new entry” (top right).
- Fill in ProjectTitel (required). The Slug will be auto-generated.
- Fill in ProjectBeschrijving, LesJaar, AantalStudenten.
- Select Opleiding (study programme), SchoolJaar (1–4), Niveau (Bachelor/Master/AD).
- Optionally add Doorlooptijd, a Foto image, Audio files, a VideoEmbed URL.
- Add Partners by clicking “Add relation” and searching for existing Partner records.
- Add Bronnen (source URLs) by clicking “+ Add a component”.
- When ready, click “Publish” (not just Save). Publishing triggers the website rebuild.

Creating a new research project

Same process as student projects, but in Content Manager → Research detail page. Key differences:

- Use ReasearcherData to add researcher name and email pairs.
- Jaar is the year of the research (integer).
- research_pages lets you link related research projects to each other.

Editing the navigation

- Go to Content Manager → Navigation (it is a Single Type — there is only one).
- Add, remove, or reorder navItems. Each has a label, url, and order number.
- Publish when done.

Managing partners

- Go to Content Manager → Partners.
- Create partner entries with a name.
- Partners are linked to student/research pages from within those pages' forms.

Generic pages

Use Content Manager → Page for any other page on the site (home page, about, contact, etc.). The home page must have slug = “home”. Each page can have multiple sections:

- **hero:** Large intro section with heading, body, image, buttons
- **text:** Body text with optional heading and image
- **cta:** Call to action with buttons
- **gallery:** Image gallery section

Draft vs Published

All content types have **draft and publish** support. A “Save” only saves a draft — it will not appear on the live website until you click “Publish”. You can safely edit and save drafts multiple times before publishing.

8. Developer Guide

Adding a new content type to Strapi

- In the Strapi admin go to Content-Type Builder → Create new collection type.
- Define the fields. Note: UID fields auto-generate slugs from a target field.
- Save. Strapi will restart and generate the boilerplate files under src/api/.
- In Settings → Roles → Public, enable find and findOne for the new type.
- The new type gets a REST endpoint at /api/<pluralName> automatically.

Adding a new route to the frontend

```
# Example: adding a /nieuws route
mkdir -p src/routes/nieuws
touch src/routes/nieuws/+page.server.js
touch src/routes/nieuws/+page.svelte
```

In `+page.server.js`, export a `load()` function that fetches from Strapi and returns data. In `+page.svelte`, destructure `let { data } = $props()` and render the data. For dynamic slugs, export an `entries()` function that returns all possible slugs.

Adding a new Svelte component

```
# Create a component folder
mkdir -p src/lib/components/MyComponent
touch src/lib/components/MyComponent/MyComponent.svelte
touch src/lib/components/MyComponent/MyComponent.module.css

# Import in a page
import MyComponent from '$lib/components/MyComponent/MyComponent.svelte';
```

Working with the Strapi REST API

All endpoints follow the pattern GET /api/<pluralName>. Key query parameters:

Parameter	Example	Effect
filters	filters[slug][$eq]=home	Filter by field value
populate	populate=*	Include all relations and components
populate (nested)	populate[sections][populate]=*	Deeply populate nested components
pagination	pagination[pageSize]=100	Fetch up to 100 records at once
publicationState	publicationState=live	Only return published records (default)

All API requests must include the header `Authorization: Bearer <token>` where the token is created in Settings → API Tokens.

Strapi TypeScript types

The folder `types/generated/` contains auto-generated TypeScript interfaces for all content types. These are regenerated automatically when you run `pnpm dev` or `pnpm build`. Do not edit these files by hand.

Code style and tooling

Tool	Config	Run with
Prettier	.prettierrc (frontend)	pnpm format
svelte-check	jsconfig.json	pnpm check
TypeScript	tsconfig.json (backend)	Checked during build

9. API Quick Reference

Base URL

```
https://your-strapi.example.com
```

Authentication

```
Authorization: Bearer <API_TOKEN>
```

Endpoints

Method + Path	Description
GET /api/pages	List all generic pages
GET /api/pages?filters[slug][$eq]=home&populate[sections][populate]=*	Fetch home page with sections
GET /api/navigation?populate=*	Fetch site navigation
GET /api/partners	List all partners
GET /api/student-detail-pages	List all student projects
GET /api/student-detail-pages?filters[Slug][$eq]=my-slug&populate=*	Fetch one student project by slug
GET /api/research-detail-pages	List all research projects
GET /api/research-detail-pages?filters[Slug][$eq]=my-slug&populate=*	Fetch one research project by slug
POST /api/rebuild	Trigger Coolify rebuild (no auth required)

Example response shape (page)

```
{
  "data": [
    {
      "id": 1,
      "documentId": "abc123...",
      "title": "Home",
      "slug": "home",
      "metaDescription": "Welcome to our site",
      "video": null,
      "sections": [
        {
          "id": 1,
          "__component": "layout.section",
          "type": "hero",
          "heading": "Welcome",
          "body": "Introductory text.",
          "image": {
            "url": "/uploads/hero.jpg", "width": 1920, "height": 1080
          },
          "buttons": [
            { "label": "Read more", "url": "/about", "style": "primary" }
          ]
        }
      ],
      "publishedAt": "2025-01-01T00:00:00.000Z"
    }
  ],
  "meta": { "pagination": { "page": 1, "pageSize": 25, "total": 1 } }
}
```

10. Troubleshooting

Problem	Likely cause	Fix
Frontend build fails with “HTTP 401”	API token missing or wrong	Check .env API_TOKEN value and ensure it is set as an env var in Coolify
Pages not showing on the site after publish	Rebuild not triggered	Check Strapi webhook config; call POST /api/rebuild manually; check Coolify deploy logs
“No home page found in Strapi”	No Page with slug “home” exists	Create a Page entry in Strapi with the slug “home” and publish it
Media images not loading	Wrong base URL prefix	Verify PUBLIC_STRAPI_API_URL matches your Strapi domain exactly (no trailing slash)
Strapi shows blank admin after deploy	Admin panel not built	Run pnpm build in Strapi or set the build command in Coolify
SQLite works locally but fails in production	SQLite not suitable for production	Set DATABASE_CLIENT=postgres in production and configure Postgres env vars
Svelte type errors on check	Missing svelte-kit sync	Run pnpm prepare first, then pnpm check
New content type not visible in API	Missing public permissions	Strapi admin → Settings → Roles → Public → enable find/findOne on the new type

11. Strapi Schema Recovery

Understanding the Problem

Strapi stores information in two places:

1. Database
The database contains:

- Articles
- Pages
- Products
- User-generated content
- Field values

Examples:

- Article Title
- Article Body
- SEO Description
- Subtitle

This data is typically stored in:

- PostgreSQL
- MySQL
- MariaDB

2. Schema Files
Strapi stores content type definitions as files inside the project.
Examples:

- src/api/article/content-types/article/schema.json
- src/api/page/content-types/page/schema.json
- src/components/shared/seo.json

These files define:

- Field names
- Field types
- Components
- Relations
- Dynamic zones

What Happens During a Rebuild

When Coolify rebuilds an application:

GitHub Repository
↓
Docker Build
↓
New Container
↓
Strapi Starts

Strapi loads schema definitions from Git.
If Git does not contain the latest schema files:
Production Container:
subtitle field exists

GitHub:
subtitle field missing
After deployment:
Strapi starts without subtitle field
The data remains in the database, but Strapi no longer knows how to display or edit it.

Symptoms

You may notice:

- Fields disappear from Content Manager
- Components disappear
- Relations stop working
- Entries cannot be edited
- Dynamic zones break
- API responses miss fields
- The database data often still exists.

Root Cause

This usually happens when someone:

1. Opens the Production Admin Panel
2. Uses Content-Type Builder
3. Creates a field
4. Does not commit generated files
5. Coolify rebuilds from GitHub

Result:
Database = New Schema
Git Repository = Old Schema

After deployment:
Strapi uses old schema

Recovery Checklist

Step 1 Check the database.
Verify content still exists.

Step 2 Check the current Git repository.
Confirm missing schema files.

Step 3 Check the running container.
Recover:
src/api
src/components

Step 4 Commit recovered files.

Step 5 Redeploy.

Step 6 Verify Content Manager functionality.

12. Glossary

Term	Meaning
Headless CMS	A CMS that stores and delivers content via API, with no built-in front-end display layer
SSG	Static Site Generation — pages are built to plain HTML files at build time, not on each request
Strapi	Open-source Node.js headless CMS used as the backend in this project
SvelteKit	Full-stack web framework built on Svelte; used here in SSG mode
Collection type	A Strapi content type that allows many entries (like a database table)
Single type	A Strapi content type with exactly one entry (e.g. Navigation)
Component	A reusable group of fields in Strapi, embedded inside a content type
Slug	A URL-safe version of a title, e.g. “My Project” → “my-project”
UID field	A Strapi field type that auto-generates a unique slug from another field
API token	A secret string used to authenticate requests to the Strapi REST API
adapter-static	A SvelteKit adapter that produces a folder of static HTML/CSS/JS files
Coolify	A self-hosted PaaS (like Heroku) used to deploy both projects
Deploy webhook	An HTTP call that triggers Coolify to start a new build and deployment
entries()	A SvelteKit function that lists all pre-renderable paths for a dynamic route
Runes	Svelte 5's new reactivity primitives: $props(), $state(), $derived(), etc.

13. Current status

Current Progress

The basic project structure has been set up, and the initial implementation for retrieving data has been completed.

Remaining tasks

- Load the P5 data properly, preferably during the build process.
- Apply the project styling.
- Add all required data fields to Strapi.
- Retrieve all required data from the source systems.
- Add and populate the website content.
- Check lighthouse

End of documentation — responsibleIT Headless CMS